

Многоверсионность. Снимки данных

Очистка и автоочистка

SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 13

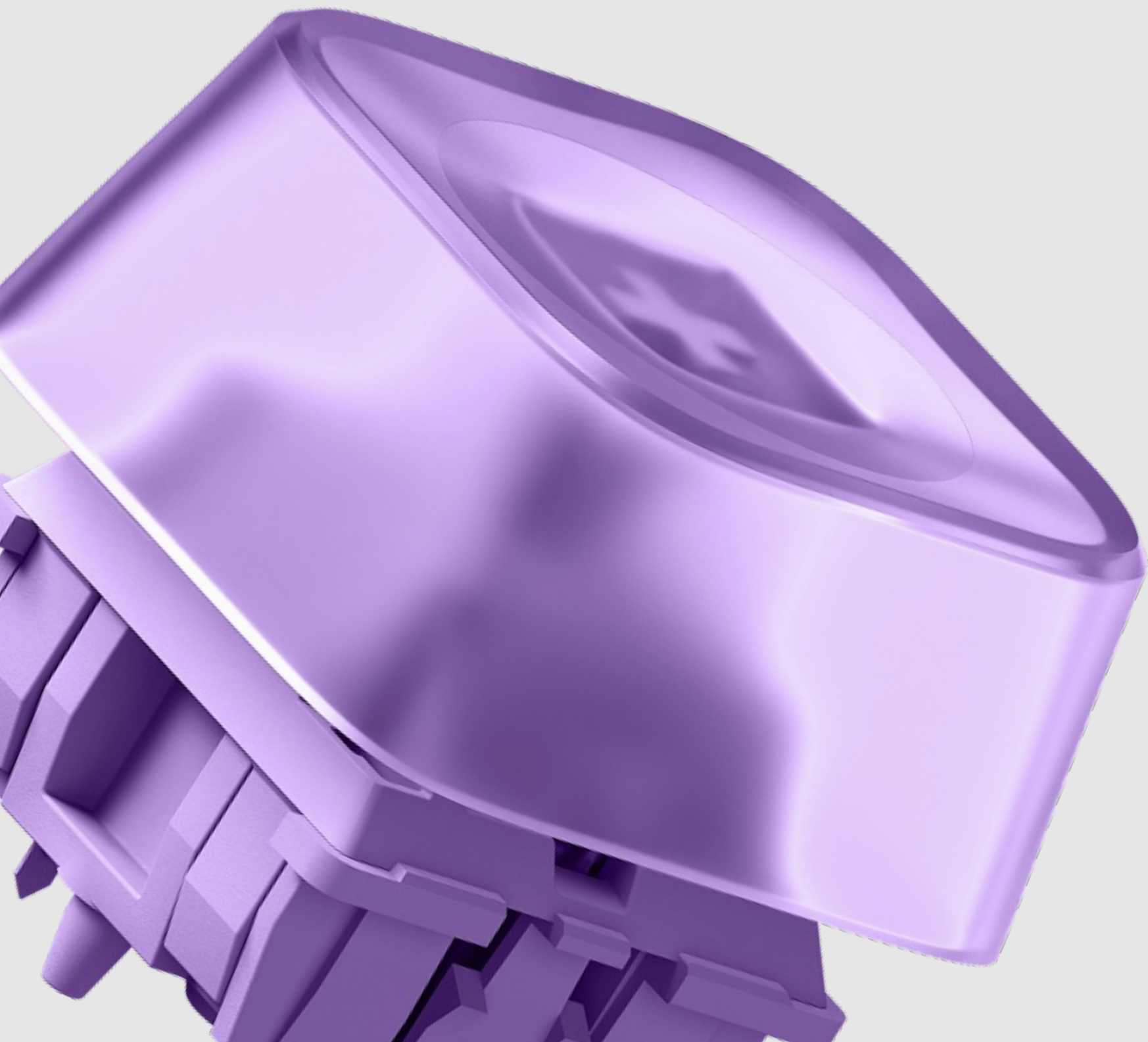
План лекции

→ Математика XID

→ VACUUM

→ Оптимизации

→ Тюнинг autovacuum



32-битный XID

Счётчик транзакций (XID) = 32 бита = 4,2 млрд уникальных значений.

Почему не 64 бита?

1. XID обязательно пишется в заголовок каждой версии строки (поля t_xmin и t_xmax).
2. Переход на 8-байтный счётчик дал бы оверхед в 8 лишних байт на каждую строку.
3. В масштабах миллиардов строк это колоссальная потеря дискового пространства и оперативной памяти.

Вывод: инженеры PostgreSQL пожертвовали размером счётчика, что породило необходимость в сложной системе сборки мусора.

Модульная арифметика



В PostgreSQL нет абсолютного времени. Время закольцовано: $(XID_A - XID_B) \bmod 2^{32}$.

Wraparound

01

Шаг 1. Транзакция $XID = 100$ создаёт строку.

02

Шаг 2. База работает под высокой нагрузкой (HighLoad), счётчик делает оборот и доходит до $XID = 2\,000\,000\,100$.

03

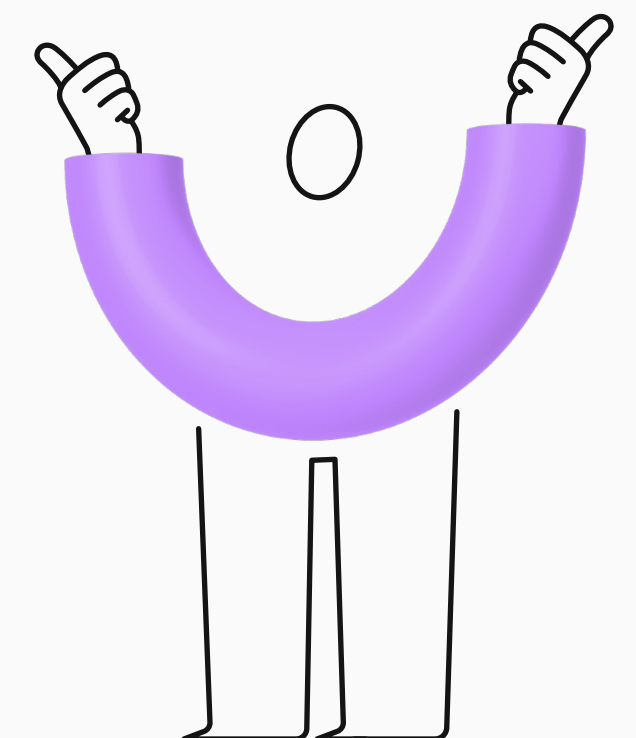
Шаг 3. Из-за кольцевой арифметики старый $XID = 100$ математически становится «будущим» для текущих транзакций.

XID stop limit



HINT: To avoid a database shutdown, execute a database-wide VACUUM... **ERROR:** database is not accepting commands to avoid wraparound data loss

1. СУБД паникует за 1 млн транзакций до точки переполнения.
2. База данных принудительно переходит в режим строгой блокировки (read-only), роняя весь пишущий production.
3. Восстановление возможно только через ручной запуск вакуума в single-user-режиме (гарантированный даунтайм).



VACUUM FREEZE

Freeze: введение системного маркера FrozenTransactionId (XID = 2).

*XID = 2 на уровне С-кода считается старше ЛЮБОЙ нормальной транзакции.
На него не действует кольцевая арифметика.



```
SELECT relname, relfrozenxid FROM pg_class;
```

```
-- Поле relfrozenxid показывает самую старую незамороженную транзакцию в таблице
```

Dead tuples

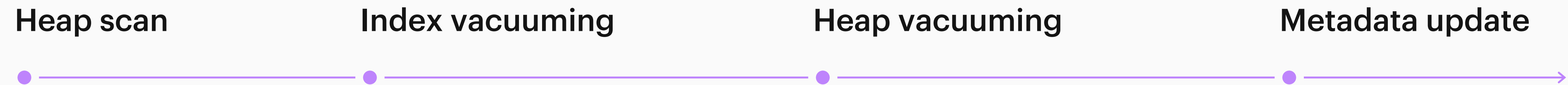
• • •

```
Tuple is DEAD IF (xmax is COMMITTED) AND (xmax < s_xmin of ALL active snapshots)
```

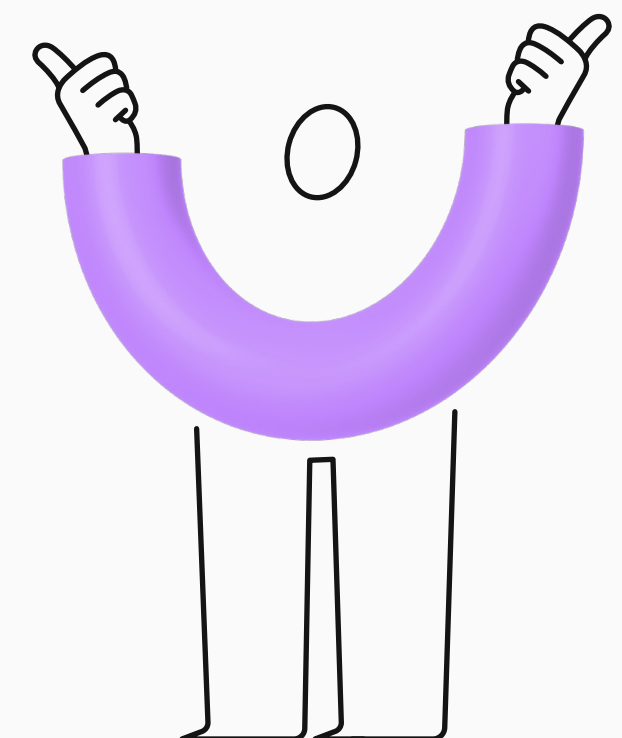
*Кортеж считается мёртвым и подлежит удалению только тогда, когда транзакция, удалившая или обновившая его, успешно зафиксирована, и в системе не осталось ни одного открытого запроса (snapshot), который математически имеет право видеть эту старую версию.

Пока жив хоть один долгий аналитический запрос, VACUUM бессилён и мусор будет копиться.

Жизненный цикл VACUUM

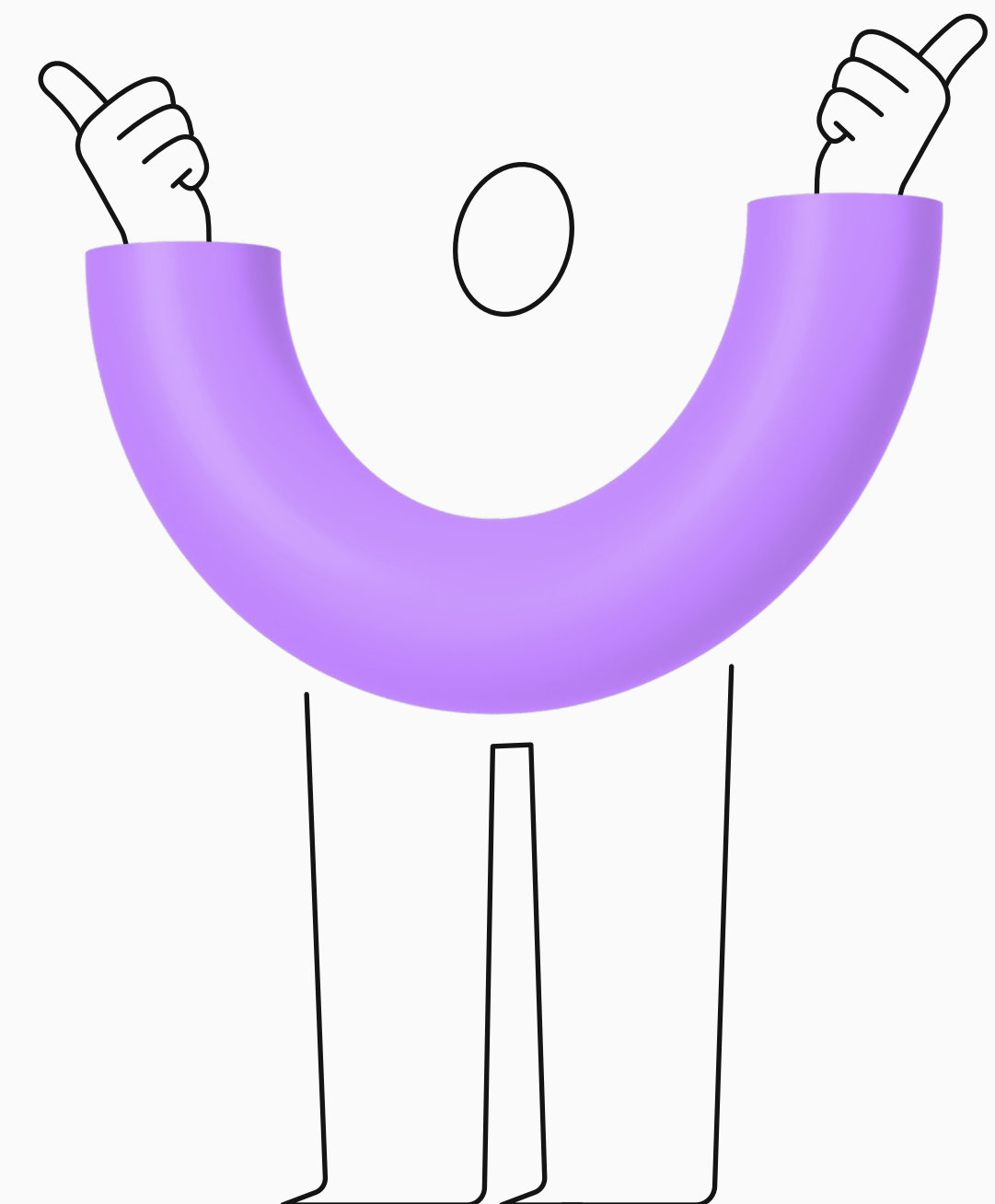


Обычный VACUUM не отдаёт дисковое пространство операционной системе, он лишь размечает место внутри файла таблицы как свободное для будущих операций INSERT и UPDATE.



Heap scan

VACUUM читает страницы кучи (heap) последовательно. Когда он находит мёртвый кортеж, то берёт его физический адрес (ctid) и записывает этот адрес во внутренний массив оперативной памяти.



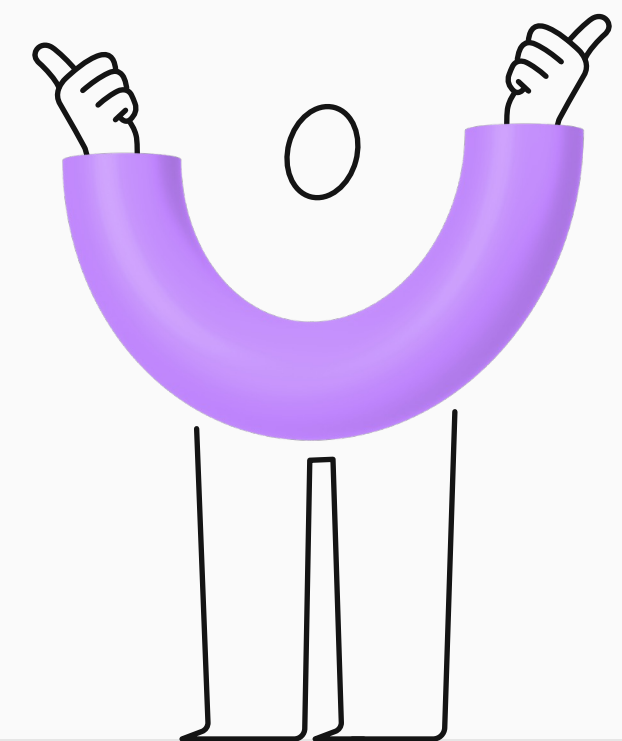
maintenance_work_mem

`maintenance_work_mem`

`ctid` = 6 byte → ~178 млн мёртвых строк в 1 ГБ.

Если мёртвых строк больше, чем влезает в память, VACUUM ставит сканирование таблицы на паузу, идёт чистить индексы, а затем возвращается и начинает новый цикл.

Недостаток `maintenance_work_mem` приводит к многократным проходам по индексам, что экспоненциально уничтожает ресурс диска (I/O).



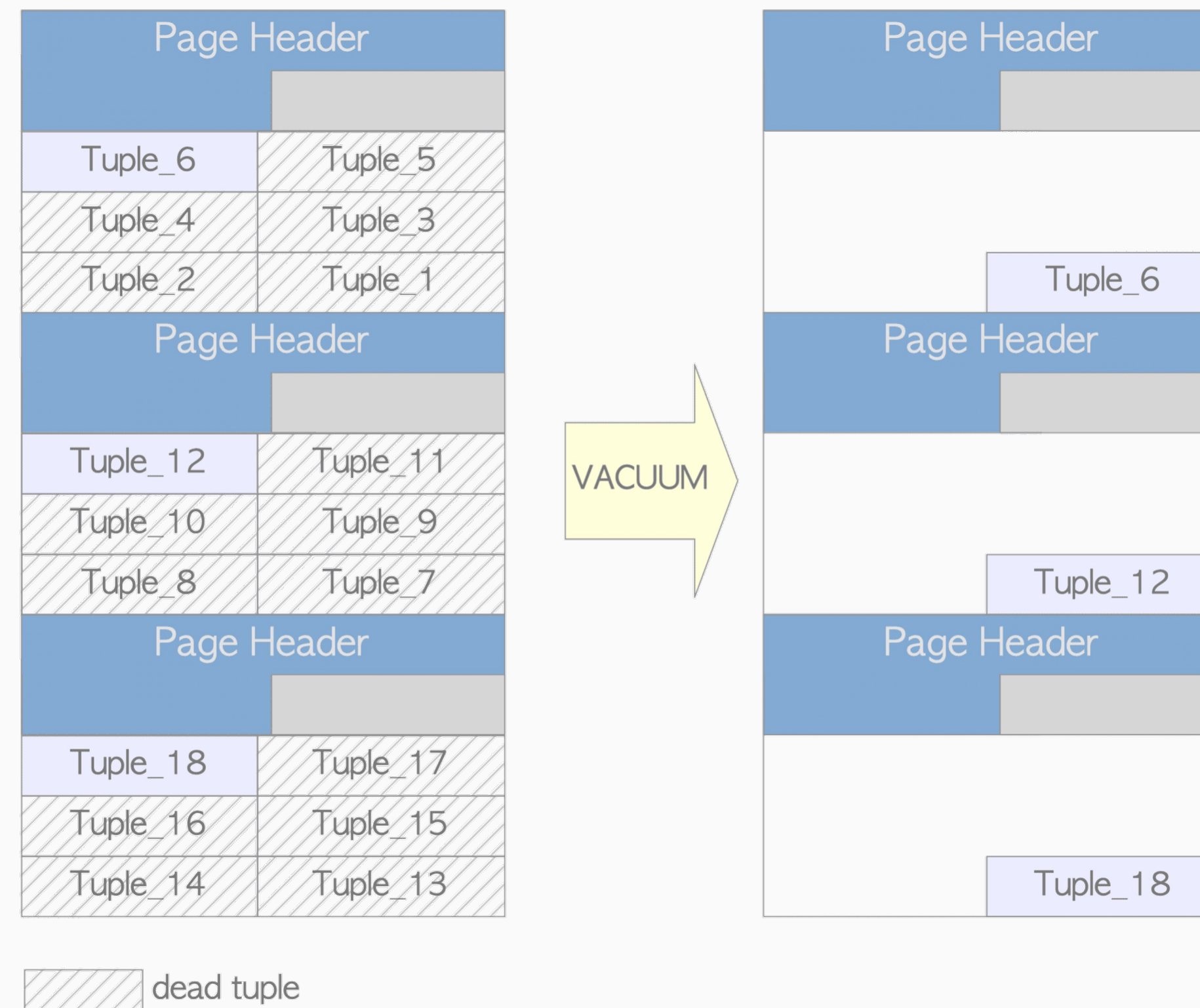
Index vacuuming

СУБД делает проход по ВСЕМ индексам, привязанным к таблице, и ищет в них ссылки на собранные ctid.

В отличие от последовательного чтения таблицы, обход B-tree-индексов — это жесточайший random I/O (случайное чтение с диска).

*Именно из-за этой фазы создание «лишних» индексов на высоконагруженной таблице замедляет не только запись, но и сборку мусора.

Heap vacuuming



Обновление метаданных

- Запись актуального количества строк в `pg_class.reltuples` для планировщика запросов
- Обновление free space map (FSM)
- Обновление visibility map (VM)

Если после очистки последние страницы файла оказались абсолютно пустыми, база «отрежет» этот хвост и вернёт место ОС.

Если же на последней странице есть хотя бы одна живая строка, файл не уменьшится ни на байт.

Пространственная навигация: free space map (FSM)

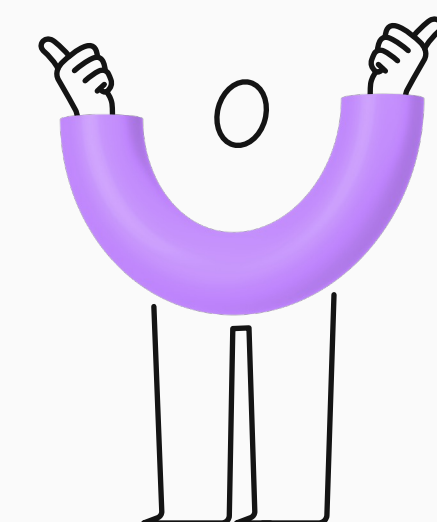
Проблема

При операции INSERT база должна найти страницу, на которой достаточно места для новой строки. Сканировать гигабайты кучи (heap) последовательно — смерть для производительности.

Решение

`_fsm` — это не просто список, а древовидная структура (n-ary tree), где каждый узел хранит информацию о максимальном свободном месте в дочерних узлах.

FSM позволяет за время $O(\log N)$ найти подходящую страницу для записи или гарантированно ответить, что нужно выделять новый блок в конце файла.

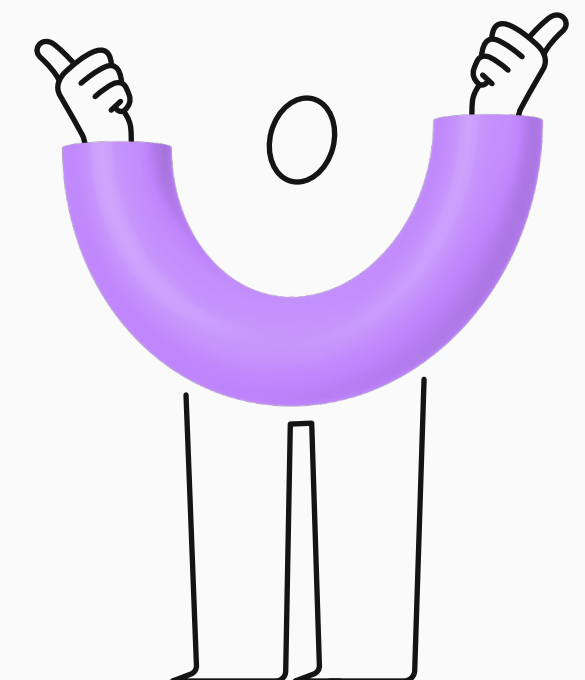


Visibility map (VM)

`_vm` — это максимально компактная битовая карта. На каждую страницу (page) размером 8 КБ выделяется ровно 2 бита.

1. Бит `all-visible` означает, что на странице нет ни одной «мёртвой» строки и все транзакции, создавшие текущие версии строк, давно закоммичены.
2. Бит `all-frozen` означает, что все строки на странице заморожены (защита от `wraparound`).

Карта видимости — это гарантия того, что данные на странице можно читать, не проверяя XID в заголовках кортежей.



Index-only scans

Проблема

Индексы (например, B-tree) в PostgreSQL НЕ хранят информацию о транзакциях (xmin/xmax). Поэтому по умолчанию, найдя нужный ключ в индексе, база обязана сходить в основную таблицу (heap scan), чтобы проверить, видна ли эта версия строки текущему снимку.

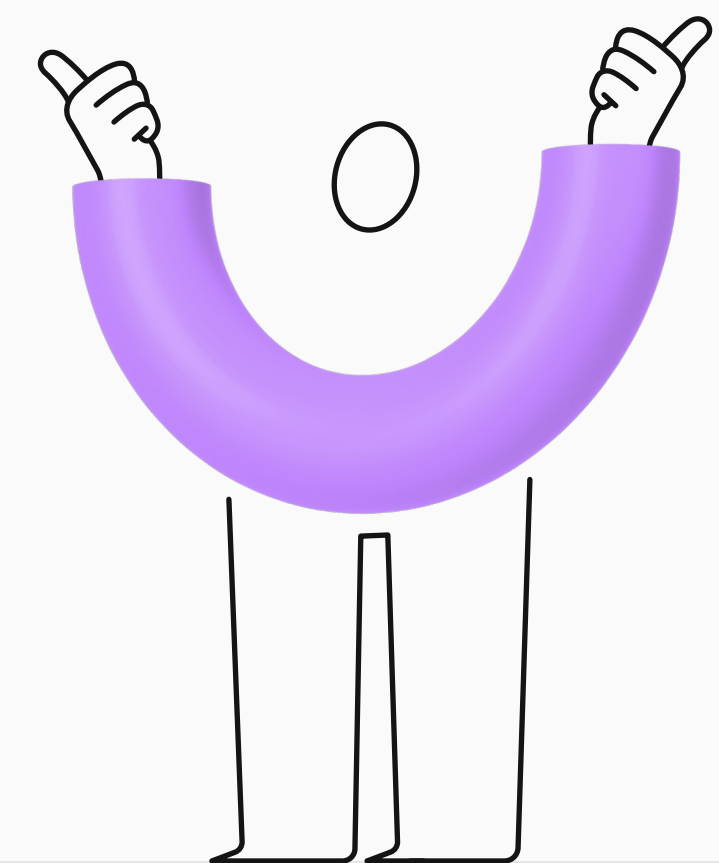
Решение

Если планировщик видит, что целевая страница помечена как all visible в карте видимости, он пропускает поход в heap. Данные отдаются прямо из индекса (index-only scan). Колоссальное снижение дискового I/O.

Симбиоз VACUUM и производительности SELECT

Бит all-visible устанавливается ТОЛЬКО процессом VACUUM.

Если на высоконагруженной таблице автовакуум не успевает работать (или отключён), бит all-visible сбрасывается при первых же UPDATE. Все index-only scans деградируют в обычные index scans. Ваши SELECT-запросы внезапно начинают тормозить в десятки раз из-за дополнительного случайного чтения кучи, хотя индексы на месте.



Проблема амплификации записи

В MVCC – UPDATE = INSERT новой версии + DELETE старой.

Поскольку новая версия строки получает новый физический адрес, СУБД обязана обновить абсолютно все индексы таблицы, чтобы они указывали на новый.

Если у вас таблица со 100 колонками и 10 индексами и вы делаете UPDATE views = views + 1, база данных выполнит 10 тяжёлых операций записи в индексы, хотя само поле views ни в одном индексе не участвует. Это смерть для диска.

HOT (heap-only tuples)

1. Обновляемое поле HE входит ни в один индекс таблицы.
2. На текущей странице данных (page размером 8 КБ) есть свободное физическое место для размещения новой версии строки.

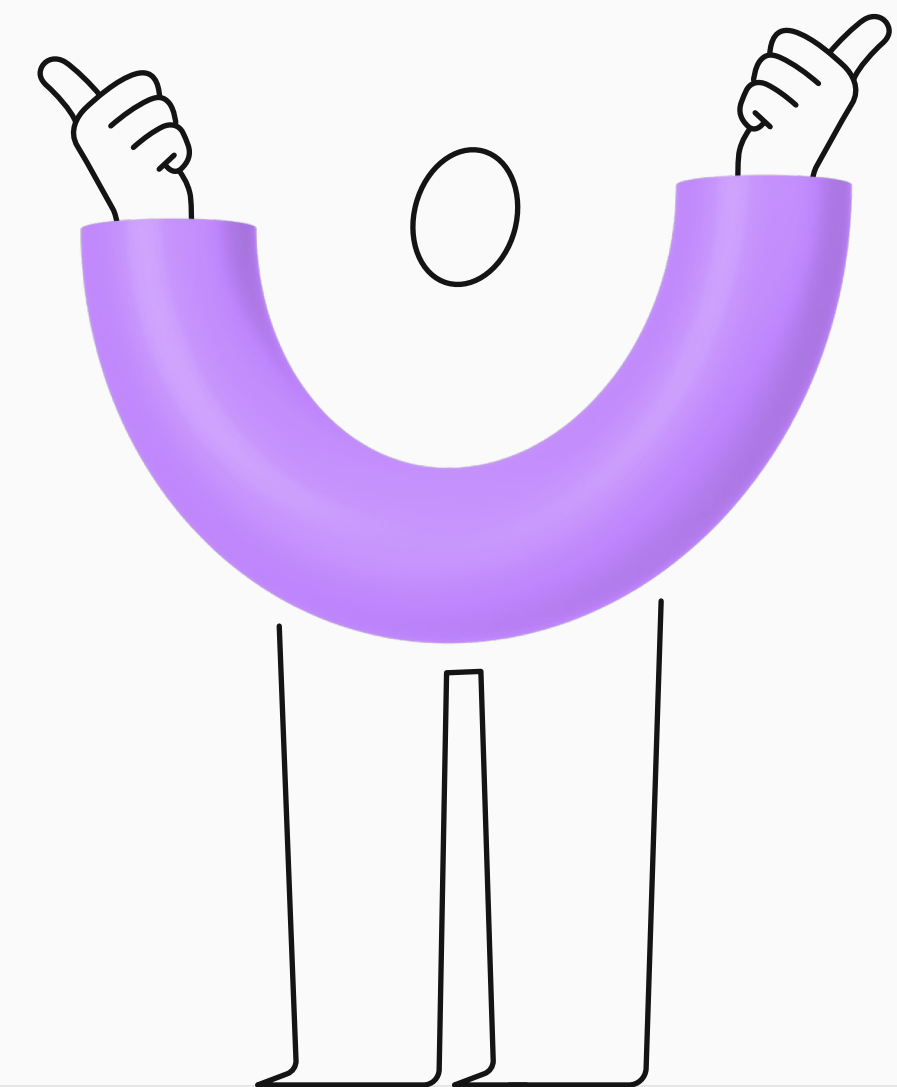
*Если условия выполнены, новая версия строки пишется на ту же страницу, а индексы HE ТРОГАЮТСЯ вообще. Операция записи становится в десятки раз дешевле.

Line pointer redirection

В куче (heap) индекс всегда ссылается на line pointer (указатель строки). При HOT-обновлении база превращает старый указатель в «перенаправление» (forwarding address).

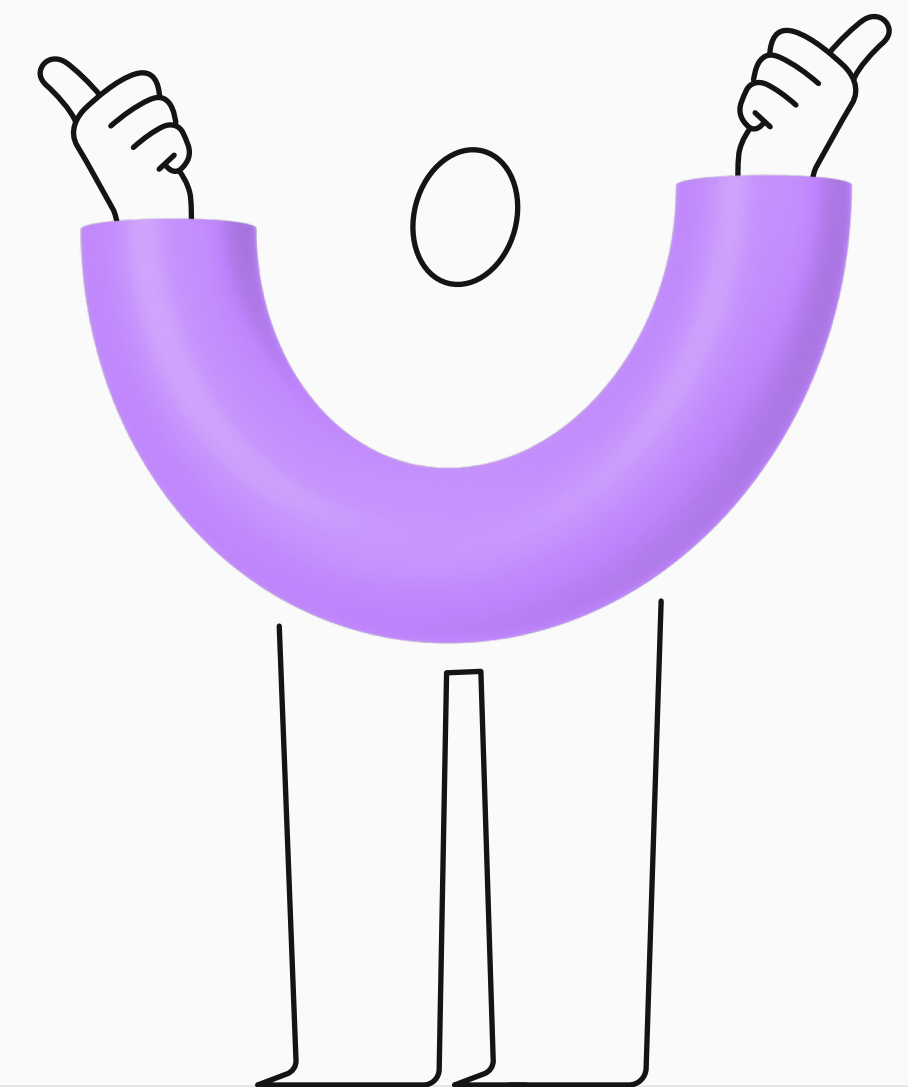
Индекс по-прежнему приходит по старому адресу, но старый указатель внутри страницы говорит: «Актуальная версия лежит чуть дальше, по указателю №5».

*Вся цепочка перенаправлений живёт строго в границах одной страницы, поэтому дополнительных чтений диска не происходит.



Микровакуум (page-level pruning)

НОТ избавляет от необходимости ждать autovacuum. Справа объяснение механики: когда любой обычный рабочий процесс читает страницу, содержащую НОТ-цепочки, он автоматически оценивает их видимость. Если процесс видит, что старые звенья цепи уже никому не видны (мертвы), он мгновенно удаляет их из страницы и сдвигает байты, освобождая место.



*Это дефрагментация на лету. Таблица чистит себя сама за счёт процессорного времени тех, кто её читает.

Тюнинг FILLFACTOR

PostgreSQL заполняет страницы данных (pages) при INSERT на 100%. Свободного места нет, а значит, условие №2 для HOT никогда не выполнится при первом UPDATE.

*Системный параметр таблиц fillfactor

```
ALTER TABLE table_name SET (fillfactor = 90) ;
```

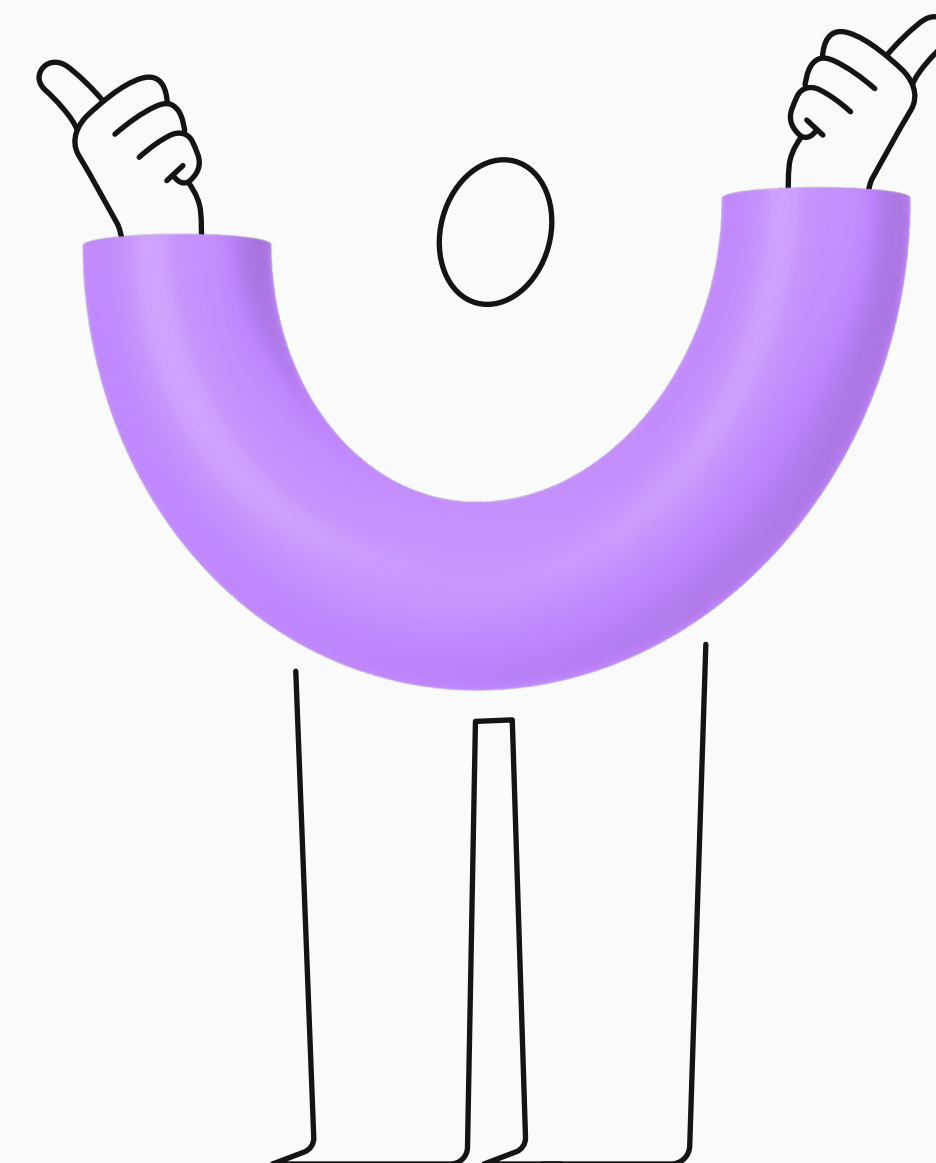
/ Эта команда заставляет базу оставлять 10% места на каждой новой странице пустым.*

*Мы жертвуем 10% дискового пространства заранее, чтобы гарантированно получить сверхбыстрые HOT-обновления в будущем */*

Архитектура демона: **launcher** и **workers**

autovacuum launcher: он просыпается раз в `autovacuum_naptime` (default = 1 m), собирает статистику по всем таблицам базы и решает, кому нужна очистка.

autovacuum workers (default `max_workers` = 3) — это рабочие процессы, которые непосредственно сканируют кучу и индексы.



Смерть от `scale factor` на сверхбольших таблицах

$50 + (0,2 \times 1\,000\,000\,000)$

1 млрд = ~1 ТБ.

К моменту запуска вакууму не хватит памяти.

Для таблиц > 10–50 ГБ всегда отключайте процентный триггер `scale_factor = 0` и фиксируйте жёсткий порог `vacuum_threshold = 100\,000` через `ALTER TABLE`.

Троттлинг ввода-вывода

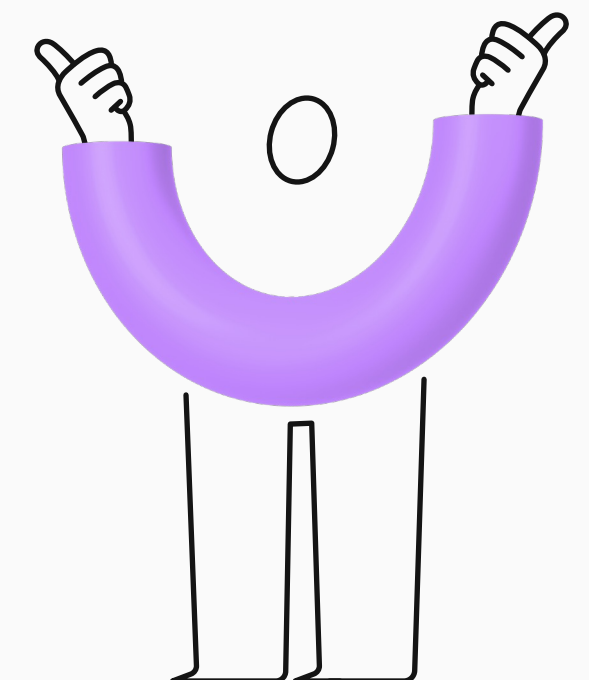
Задача

Фоновый процесс очистки не имеет права убивать latency пользовательских SELECT/UPDATE-запросов.

Решение

База «штрафует» вакуум за операции: чтение из кеша `cost_page_hit = 1`, чтение с диска `cost_page_miss = 10`, запись грязной страницы `cost_page_dirty = 20`.

Чтобы заставить вакуум работать агрессивнее на современных быстрых NVMe/SSD, нужно увеличивать `cost_limit` и уменьшать `cost_delay`.



Метаданные и иллюзия сжатия файла



VACUUM FULL и альтернативы

(1) Create new table file

Old table file

Page Header	
Tuple_6	
Page Header	
Tuple_12	
Page Header	
Tuple_18	

New table file

Page Header	

(2) Copy live tuples to new table file

Old table file

Page Header	
Tuple_6	
Page Header	
Tuple_12	
Page Header	
Tuple_18	

New table file

Page Header	
	123
Tuple_18	
Tuple_6	

(3) Remove old file, update FSM, VM, etc.

New table file

Page Header	
	123
	Tuple_18
Tuple_12	Tuple_6

 dead tuple

Мониторинг и архитектурные догматы

Инструмент

Описание

pg_stat_user_tables

Мониторинг количества мёртвых строк и того, когда демон приходил последний раз

pg_stat_progress_vacuum

На какой именно фазе (скан кучи, зачистка индексов) сейчас находится вакуум, позволяя поймать зависание

pgstattuple

Утилита для точного побайтового подсчёта фрагментации (bloat) внутри файлов

Резюме

→ Никогда не отключайте фоновый autovacuum.

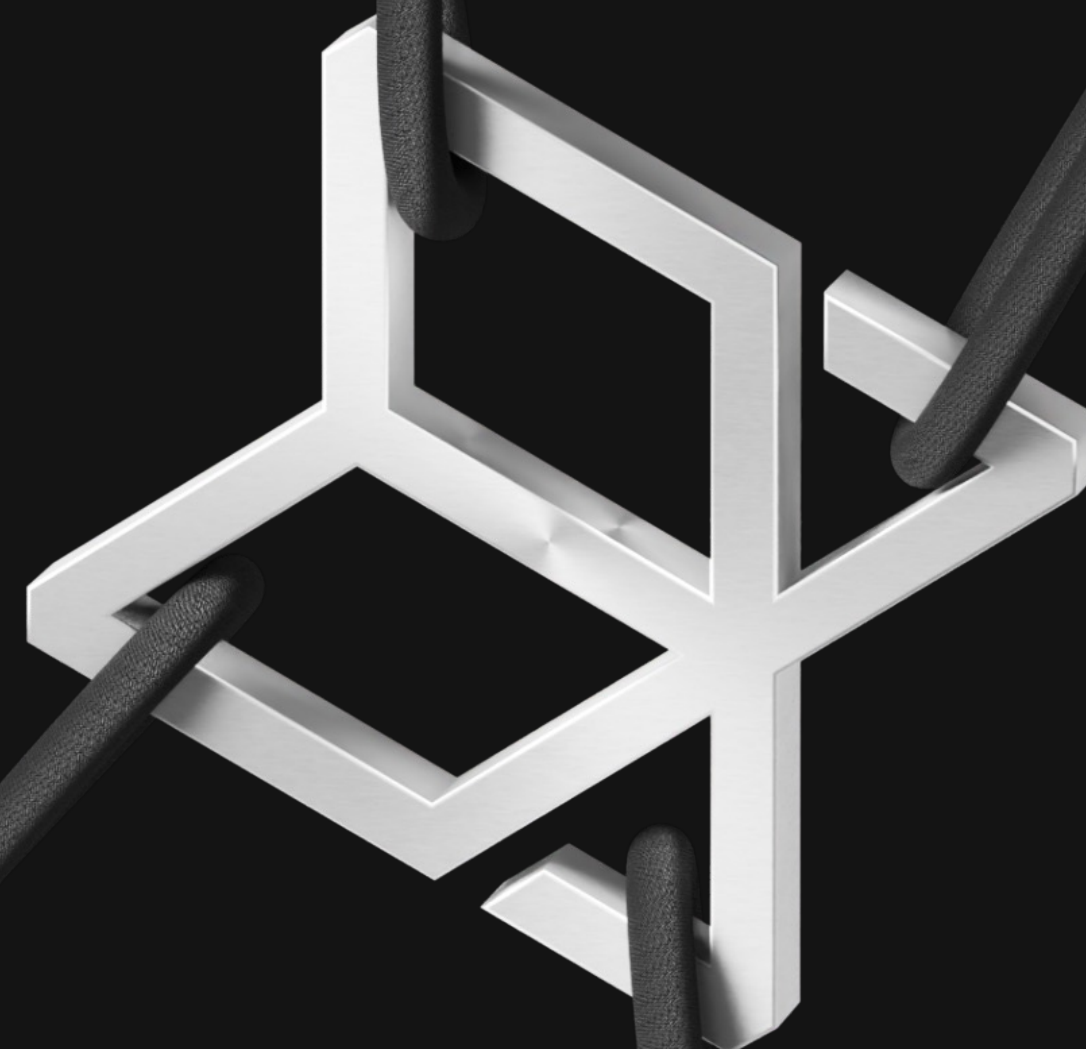
→ Для таблиц размером более 10–50 ГБ всегда сбрасывайте `autovacuum_vacuum_scale_factor = 0`.

→ Заранее закладывайте пространство под HOT-обновления с помощью `filfactor = 90`.

→ Настройте критический алерт на возраст `relfrozenxid`.



Вопросы



Минутка обратной связи

